Q1. Explain the difference between Gradient Descent, Stochastic Gradient Descent (SGD), and Mini-batch Gradient Descent.
Also compare them in terms of speed, stability, and practical usage.

1️⃣ Gradient Descent (Batch Gradient Descent)

🔹 ধারণা

Gradient Descent এ পুরো **dataset** একসাথে ব্যবহার করে model-এর weight update করা হয়।

🔹 কীভাবে কাজ করে

1. সব training data নেওয়া হয়
2. total loss calculate করা হয়
3. gradient বের করা হয়
4. weight update করা হয়

🔹 উদাহরণ

ধরো dataset = **10,000 data**

তাহলে,

➡️ সব 10,000 data একসাথে use করবে
➡️ তারপর weight update করবে
➡️ ১ epoch এ ১ বার update

🔹 বৈশিষ্ট্য

✅ Stable
✅ Smooth learning
❌ Slow
❌ Large dataset এ time বেশি লাগে

2️⃣ Stochastic Gradient Descent (SGD)

🔹 ধারণা

SGD এ একটি **data** ব্যবহার করে **weight update** করা হয়

🔹 কীভাবে কাজ করে

1 data নাও
loss বের করো
weight update করো
পরের data

🔹 উদাহরণ

Dataset = 10,000

➡️ 1 data → update
➡️ 1 data → update
➡️ 1 data → update

মোট update = **10,000**

🔹 বৈশিষ্ট্য

✅ খুব fast
✅ দ্রুত শিখে
❌ Noise বেশি
❌ Stable না
❌ oscillation করে

৩️⃣ Mini-batch Gradient Descent

🔹 ধারণা

এখানে ছোট ছোট **batch** ব্যবহার করা হয়

যেমন:

- 16
- 32
- 64
- 128

🔹 উদাহরণ

Dataset = 10,000
Batch size = 100

10,000 / 100 = 100 batch

➡️ 100 বার update হবে

🔹 বৈশিষ্ট্য

✅ Fast
✅ Stable
✅ Efficient
✅ GPU friendly
✅ Deep learning এ বেশি ব্যবহার হয়

📊 Comparison (Speed, Stability, Practical Usage)

Method	Speed	Stability	Practical Usage
Gradient Descent	Slow	Very Stable	Small dataset
SGD	Very Fast	Unstable	Online learning
Mini-batch	Fast	Stable	Deep Learning

⚡ Speed Comparison

Gradient Descent

- Slow
- সব data একসাথে
- time বেশি লাগে

SGD

- Very fast
- সাথে সাথে update

Mini-batch

- Fast
- Balanced

✅ Best speed

📈 Stability Comparison

Gradient Descent

✔ Smooth learning
✔ Stable

SGD

❌ Jump করে
❌ Noise বেশি

Mini-batch

✔ Less noise
✔ Stable
✔ Efficient

🧠 Practical Usage

Gradient Descent

- Small dataset
- Theory
- Research explanation

SGD

- Online learning
- Streaming data
- Real-time system

Mini-batch Gradient Descent

সবচেয়ে বেশি ব্যবহার হয়:

- Deep Learning
- Neural Network
- PyTorch
- TensorFlow
- CNN
- RNN

Batch size:

- 32
- 64
- 128

🎯 Short Final Answer (Exam Style)

Gradient Descent পুরো dataset ব্যবহার করে weight update করে, তাই slow কিন্তু stable।

Stochastic Gradient Descent (SGD) একটি data ব্যবহার করে update করে, তাই fast কিন্তু unstable।

Mini-batch Gradient Descent ছোট batch ব্যবহার করে update করে, তাই fast এবং stable এবং deep learning এ সবচেয়ে বেশি ব্যবহার হয়।

Q2. Why does Momentum help reduce oscillation and speed up learning?
Explain using the idea of zig-zag movement and velocity.

Momentum কী?

Momentum হলো এমন একটি technique যা **previous gradient (**আগের **update)** ব্যবহার করে weight update করে, যাতে learning দ্রুত এবং smooth হয়। 🚀

মানে,

> Model শুধু current gradient দেখে না, আগের direction-ও মনে রাখে।

🔷 Problem Without Momentum (Zig-Zag Movement)

ধরো আমরা **Gradient Descent** দিয়ে model train করছি।

তখন কী হয়?

- Gradient একবার ডানে যায়
- আবার বামে যায়
- আবার ডানে যায়
- আবার বামে যায়

এভাবে **zig-zag movement** হয় ⚡

কেন এমন হয়?

কারণ:

- gradient direction বারবার change হয়
- loss surface uneven হয়
- steep direction এ jump করে

Result

❌ Oscillation হয়
❌ Slow convergence
❌ Time বেশি লাগে

🔹 Zig-Zag Movement Example

ধরো valley এর মধ্যে ball নামছে।

Without momentum:

➡️ ডানে লাফ
➡️ বামে লাফ
➡️ ডানে লাফ
➡️ বামে লাফ

Slowly নিচে নামবে।

এটাই zig-zag learning।

🔹 Momentum কীভাবে কাজ করে

Momentum previous gradient কে **velocity** হিসেবে use করে।

Idea

Velocity = previous direction

New update = current gradient + previous velocity

মানে,

➡️ আগের direction মনে রাখে
➡️ একই direction এ speed বাড়ায়
➡️ zig-zag কমায়

🔹 Velocity Concept

Momentum মনে করে model একটা **rolling ball**।

যেমন:

- Ball hill থেকে নামছে
- speed বাড়ছে
- একই direction এ যাচ্ছে

এটাই velocity।

Result

✔ Smooth movement
✔ Less oscillation
✔ Faster convergence

🔷 Zig-Zag Reduction

Without Momentum:

↘↙↘↙↘↙

Zig-zag

With Momentum:

↓↓↓↓↓↓

Straight movement

🔷 Why Oscillation Reduces

কারণ:

- Momentum sideways movement কমায়
- Same direction accumulate করে
- Noise cancel করে

ফলে

✔ Smooth path
✔ Less zig-zag
✔ Stable learning

🔹 Why Speed Increases

Momentum:

- Previous gradient add করে
- Direction maintain করে
- Acceleration দেয়

ফলে

✔ দ্রুত valley তে পৌঁছে
✔ Faster convergence
✔ কম epoch লাগে

🔹 Real-Life Example

Without Momentum

একটা বল rough পাহাড়ে নামছে

- লাফাচ্ছে
- ধীরে নামছে

With Momentum

একটা বল smooth ভাবে গড়িয়ে নামছে

- speed বাড়ছে
- দ্রুত নিচে যাচ্ছে

📊 Summary

Without Momentum	With Momentum
Zig-zag movement	Smooth movement
Oscillation বেশি	Oscillation কম
Slow learning	Fast learning
Direction change বেশি	Direction stable
Time বেশি লাগে	Time কম লাগে

🎯 Exam Style Answer

Momentum oscillation কমায় এবং **learning speed** বাড়ায় কারণ এটি **previous gradient** কে **velocity** হিসেবে ব্যবহার করে। **Gradient Descent** এ **zig-zag movement** হয়, যার ফলে **oscillation** বেশি হয় এবং **learning slow** হয়। **Momentum** আগের **direction** মনে রেখে একই **direction** এ **movement** করে, ফলে **zig-zag** কমে যায় এবং **smooth path** তৈরি হয়। এর ফলে **model** দ্রুত **convergence** করে এবং **learning speed** বাড়ে। ✅

Q3. What is the main problem of AdaGrad?
Explain how it affects the learning rate during training.

AdaGrad-এর **main problem** হলো **training** চলতে চলতে **learning rate** ধীরে ধীরে খুব ছোট হয়ে যায়। **AdaGrad past gradient-**এর **square sum accumulate** করে, যার ফলে **denominator** বড় হতে থাকে এবং **learning rate** কমতে থাকে। **Training** এর শুরুতে **learning rate** বেশি থাকে এবং **model** দ্রুত শিখে, কিন্তু পরে **learning rate** খুব ছোট হয়ে যায় এবং **model** আর **update** করতে পারে না। এর ফলে **learning slow** হয়ে যায় এবং **convergence problem** তৈরি হয়।

Solution

এই **problem solve** করার জন্য নতুন **optimizer** এসেছে:

- **RMSProp**
- **Adam**

কারণ তারা **past gradient** পুরোটা **accumulate** করে না, বরং **moving average** ব্যবহার করে।

Q4. How does RMSProp solve the problem of AdaGrad?
Explain the difference between full accumulation and moving average.

RMSProp AdaGrad-এর **problem solve** করে **moving average** ব্যবহার করে। **AdaGrad past gradient-**এর **full accumulation** করে, যার ফলে **learning rate** ধীরে ধীরে খুব ছোট হয়ে যায়। **RMSProp** সব **gradient** জমা না করে **recent gradient-**এর **moving average** ব্যবহার করে, ফলে **denominator stable** থাকে এবং **learning rate** খুব ছোট হয় না। তাই **RMSProp faster learning** এবং **better convergence** প্রদান করে। **Full accumulation-**এ সব **past gradient** জমা হয়, কিন্তু **moving average-**এ **recent gradient** বেশি গুরুত্ব পায় এবং পুরনো **gradient** ধীরে ধীরে কমে যায়। ✅

learning rate = η / √(past gradient sum)

Q5. Why is Adam considered a combination of Momentum and RMSProp?
Explain in simple terms.

Adam কী?

Adam (Adaptive Moment Estimation) হলো এমন একটি optimizer যা

- **Momentum-**এর **velocity idea** ব্যবহার করে
- **RMSProp-**এর **adaptive learning rate** ব্যবহার করে

👉 তাই এটাকে **Momentum + RMSProp combination** বলা হয়।

🔹 Adam কীভাবে Momentum ব্যবহার করে

Momentum কী করে?

- Previous gradient মনে রাখে
- Direction smooth করে
- Zig-zag কমায়
- Speed বাড়ায়

Adam-এ কী হয়?

Adam **past gradient-**এর **moving average** রাখে

মানে,

➡️ আগের direction মনে রাখে
➡️ smooth movement করে
➡️ oscillation কমায়

✔ এটা Momentum-এর কাজ

🔹 Adam কীভাবে RMSProp ব্যবহার করে

RMSProp কী করে?

- past gradient square-এর moving average রাখে
- learning rate adaptive করে
- learning rate ছোট হয়ে যেতে দেয় না

Adam-এ কী হয়?

Adam:

➡️ gradient square-এর moving average রাখে
➡️ learning rate adjust করে
➡️ stable update দেয়

✔ এটা RMSProp-এর কাজ

🔹 Adam-এর ভিতরে দুইটা জিনিস থাকে

1️⃣ Momentum Part

past gradient average

➡️ direction maintain করে
➡️ speed বাড়ায়

2️⃣ RMSProp Part

gradient square average

➡️ learning rate control করে
➡️ stable learning দেয়

🔹 Simple Concept

Adam দুইটা জিনিস একসাথে করে:

Momentum

Direction smooth করে

RMSProp

Learning rate control করে

➡️ তাই Adam powerful optimizer

🔹 Real-Life Example

ধরো তুমি পাহাড় থেকে নামছো।

Momentum

তোমার speed maintain করে

➡️ smooth নামছো

RMSProp

তুমি কোথায় fast বা slow হবে সেটা control করে

➡️ safe নামছো

Adam

দুইটা একসাথে

➡️ smooth + controlled + fast

🔹 Comparison

Feature	Momentum	RMSProp	Adam
Previous gradient	Yes	No	Yes
Gradient square	No	Yes	Yes

Learning rate control	No	Yes	Yes
Smooth movement	Yes	Yes	Yes
Fast convergence	Medium	Good	Best

🔹 Why Adam is Combination

কারণ Adam:

- Momentum-এর **gradient moving average** ব্যবহার করে
- RMSProp-এর **gradient square moving average** ব্যবহার করে

➡️ তাই Adam = Momentum + RMSProp

🎯 Exam Style Answer

Adam optimizer Momentum এবং **RMSProp-**এর **combination** হিসেবে বিবেচিত হয় কারণ এটি **past gradient-**এর **moving average** ব্যবহার করে **direction smooth** করে **(Momentum-**এর মতো**)** এবং **gradient square-**এর **moving average** ব্যবহার করে **adaptive learning rate** তৈরি করে **(RMSProp-**এর মতো**)**। ফলে **Adam oscillation** কমায়**, learning rate control** করে এবং দ্রুত **convergence** প্রদান করে। তাই **Adam** কে **Momentum** এবং **RMSProp-**এর **combination** বলা হয়।

Q6. Given:
w = 5, gradient (dw) = 2, learning rate = 0.1
Calculate the updated weight using SGD.

🎯 Exam Style Answer

Using SGD update rule,

$$
\begin{aligned}
w_{new} &= w - \eta \times dw \\
&= 5 - (0.1 \times 2) \\
&= 5 - 0.2 \\
&= 4.8
\end{aligned}
$$

So, the updated weight is 4.8. ✅

Q7. Which optimizer would you choose for:
a) sparse data (like NLP tasks)
b) deep neural networks
Explain your choices.

a) Sparse data (like NLP tasks)

✅ Recommended Optimizer: AdaGrad

🔹 কেন **AdaGrad?**

Sparse data (যেমন NLP, text data) এ অনেক feature **0** থাকে এবং কিছু feature খুব কম update হয়।

AdaGrad এখানে ভালো কাজ করে কারণ:

- প্রতিটি parameter-এর জন্য আলাদা learning rate দেয়
- কম update হওয়া feature-কে বেশি learning rate দেয়
- বেশি update হওয়া feature-কে কম learning rate দেয়
- rare words বা sparse feature ভালোভাবে learn করতে পারে

🔹 **Example**

NLP task:

- word embeddings
- text classification
- sentiment analysis

এখানে অনেক rare word থাকে।

AdaGrad:

➡️ rare word-কে বেশি গুরুত্ব দেয়
➡️ sparse data handle করে
➡️ better learning দেয়

🎯 Answer (Exam Style)

Sparse data (NLP tasks) এর জন্য **AdaGrad optimizer** ব্যবহার করা ভালো কারণ এটি প্রতিটি **parameter-**এর জন্য **adaptive learning rate** ব্যবহার করে। **Sparse data** তে অনেক **feature** কম **update** হয়**,** তাই **AdaGrad** সেই **feature-**গুলোর **learning rate** বেশি রাখে এবং **rare feature** ভালোভাবে **learn** করতে পারে। এজন্য **NLP** এবং **text data** তে **AdaGrad** উপযুক্ত **optimizer।**

Deep Neural Networks

✅ Recommended Optimizer: Adam

🔹 কেন **Adam?**

Deep Neural Network এ:

- অনেক layer থাকে
- gradient small হয়
- oscillation হয়
- learning slow হতে পারে

Adam এখানে ভালো কারণ:

- Momentum ব্যবহার করে → zig-zag কমায়
- RMSProp ব্যবহার করে → learning rate control করে
- fast convergence দেয়
- stable learning দেয়
- deep learning এ widely used

🔹 Example

Deep Neural Network:

- CNN
- RNN
- Transformer
- Image classification
- Speech recognition

এখানে Adam সবচেয়ে বেশি ব্যবহার হয়।

🎯 Answer (Exam Style)

Deep Neural Networks এর জন্য **Adam optimizer** ব্যবহার করা ভালো কারণ এটি **Momentum** এবং **RMSProp-**এর **combination** ব্যবহার করে। এটি **oscillation** কমায়**, adaptive learning rate** দেয় এবং **fast convergence** প্রদান করে। **Deep neural networks** এ অনেক **layer** এবং **complex structure** থাকে**,** তাই **Adam stable** এবং **efficient learning** নিশ্চিত করে। এজন্য **deep learning** এ **Adam** সবচেয়ে বেশি ব্যবহৃত **optimizer**।

If your model is learning very slowly, which method would you try:
Momentum, RMSProp, or Adam?
Explain your answer.

🔹 কেন **Adam** ব্যবহার করবো?

যদি model খুব ধীরে শিখে **(learning very slowly)**, তাহলে **Adam optimizer** ব্যবহার করা সবচেয়ে ভালো।

কারণ Adam:

- Momentum ব্যবহার করে → speed বাড়ায় 🚀
- RMSProp ব্যবহার করে → learning rate control করে
- fast convergence দেয়
- stable learning দেয়

তাই slow learning problem দ্রুত solve করতে পারে।

🔹 Momentum কেন একা যথেষ্ট না

Momentum:

- zig-zag কমায়
- speed বাড়ায়
- direction smooth করে

কিন্তু

❌ learning rate control করে না
❌ adaptive না

তাই slow learning পুরোপুরি solve করতে পারে না।

🔹 RMSProp কেন একা যথেষ্ট না

RMSProp:

- adaptive learning rate দেয়
- slow learning কমায়

কিন্তু

❌ momentum effect নেই
❌ direction smooth করে না

তাই speed পুরো optimize হয় না।

🔹 Adam কেন best

Adam = Momentum + RMSProp

মানে:

- speed বাড়ায়
- learning rate adjust করে
- oscillation কমায়

- fast convergence দেয়

➡️ slow learning problem efficiently solve করে

📊 Comparison

Optimizer	Speed	Learning Rate Control	Best for Slow Learning
Momentum	Medium	No	❌
RMSProp	Good	Yes	⚠️
Adam	Very Fast	Yes	✅ Best

🎯 Exam Style Answer

If a model is learning very slowly, Adam optimizer should be used. কারণ **Adam Momentum** এবং **RMSProp-**এর **combination** ব্যবহার করে, যা **learning speed** বাড়ায় এবং **adaptive learning rate** প্রদান করে। **Momentum zig-zag** কমায় এবং **RMSProp learning rate control** করে, ফলে **Adam fast convergence** এবং **stable learning** নিশ্চিত করে। তাই **slow learning problem** হলে **Adam** সবচেয়ে উপযুক্ত **optimizer**। ✅